starting with the letter 'i' is initialised to a value when the instrument is started, and does not usually change. The letter 'a' indicates an audio rate variable and the letter 'k' indicates a control rate variable.

There are also some special 'p' variables or parameters that send values from the score to the orchestra—p1, p2 and p3 are the instrument's number, start time and their duration. The variables p4, p5, p6, etc, are flexible. They are used here for amplitude, frequency and the waveform table.

Now, add a call to ***add_score*** to the ***tutorial.py*** as follows:

```
csound.setCommand('csound -Wo tutorial.wav temp.orc temp.sco')
add_score(csound)
# Export the .orc and .sco file for performance
```

Now, code an *add_score* method:

```
def add_score(csound):
  sarega = [130.8, 146.8, 164.8, 174.6, 195.0, 220.0, 246.9, 261.6]
  for time in range(8):
    csound.addNote(1, time, 1, 8000  , sarega[time], 1)
```

Your instrument will play the notes. Let us improve the instrument and put an envelope in each event. Modify the instrument definition in *tutorial.csd* as follows:

```
  instr 1
iamp = p4
ifqc = p5
itabl1 = p6
kamp linseg 0, .2, 1, .2, .8, p3-.5, .8, .2, 0
aout oscil iamp, ifqc, itabl1
  outs aout*kamp, aout*kamp
  endin
```

You can define an envelope using the opcode *linseg*, which represents the starting amplitude followed by pairs of time intervals and the amplitude at the end of the interval.

The control variable ***kamp*** starts with 0, rises to 1 in .2 seconds, then drops to .8 in the next .2 seconds, retaining that value until .2 seconds before the end. In the last .2 seconds, the value drops from .8 to 0. As you can imagine, an instrument can be programmed to generate pretty complex sounds for each event. You can get an idea of the programming possibilities by playing two frequencies close to each other. Replace the *add_score* method in the *tutorial.py* with:

```
def add_score(csound):
  sarega = [130.8, 146.8, 164.8, 174.6, 195.0, 220.0, 246.9, 261.6]
  for time in range(8):
    csound.addNote(1, time, 1, 8000 ,  sarega[time], 1)
    csound.addNote(1, time, 1, 8000 ,  sarega[time] + 5, 1)
```

Not surprisingly, you should hear beats.

Musicians do not work with frequencies. They work with octaves. So, let us define another instrument that uses a converter opcode to convert a number into a frequency. The whole number represents the octave, and the decimal part the semitone. So, at the instruments section in *tutorial.csd*, add the following:

```
  instr 2
iamp = p4
ifqc = cpspch(p5)
itabl1 = p6
kamp linseg 0, .2, 1, .2, .8, p3-.5, .8, .2, 0
asigl oscil iamp, ifqc*.999, itabl1
asigr oscil iamp, ifqc*1.001, itabl1
  outs asigl*kamp, asigr*kamp
  endin
```

Notice that the definition has a different oscillator definition for the right and left channels. In the score section, include a second waveform table:

```
f2  0   16384   10  1 .5  .3333
```

This table is also a sine wave but includes the first and second harmonics with amplitudes that are a half and a third of the primary wave.

Now, modify the ***add_score*** method in tutorial.py to use both instruments:

```
def add_score(csound):
  sarega = [130.8, 146.8, 164.8, 174.6, 195.0, 220.0, 246.9, 261.6]
  pitch  = [8.00, 8.02, 8.04, 8.05, 8.07, 8.08, 8.11, 9.00]
  for time in range(8):
    csound.addNote(1, time, 1, 8000  , sarega[time], 1)
    csound.addNote(2, time, 2, 8000, pitch[time], 1)
```

The second instrument plays the same tones but at an octave higher. It also uses the second waveform table. You can explore various programming possibilities—like varying amplitudes, varying durations and varying the relative start time of instruments.

As you would have noticed, there is no constraint on the size of the orchestra you can single-handedly create. You can find some simple drum instruments at ***www.csounds.com/ezine/autumn1999/synthesis/index.html***. So go ahead, try them and, may be, create a synthetic tabla!

Next month, we will explore playing around Soundfonts and Csound to create sounds. END

By: Dr. Anil Seth

The author is a consultant by profession and can be reached at seth.anil@gmail.com